Assignment 2: Text Sequences with Word Embeddings, Autocompletion, and Attention

Section 1: Data Collection, Preprocessing & Dynamic Sequence Batching [10 Marks]

Task 1.1: Data Collection [2 Marks]

- Load the dataset using NLTK's Gutenberg corpus specifically targeting **carroll-alice.txt**.
- Split the raw text into sentences using NLTK's sentence tokenizer (**sent_tokenize**). Note: Download both **punkt** and **punkt_tab** packages.
- Print the total sentence count, total raw word count, and raw unique vocabulary size.

Task 1.2: Tokenization, Truncation, and Dynamic Sequence Generation [5 Marks]

- Clean raw sentences by lowercasing and stripping non-alphanumeric characters except basic punctuation marks (period, comma, exclamation point, question mark).
- Tokenize each cleaned sentence into a list of word strings.
- Build **word2idx** and **idx2word** dictionaries using a frequency table, reserving at index 0 and at index 1.
- Prefix Generation with Truncation: For every sentence's token index list [i1, i2, ..., in], generate training pairs for every prefix step up to n -1:
  - (X = [i1], y = i2)
  - (X = [i1, i2], y = i3)
  - ...
- Max Sequence Cap (**MAX_LEN = 20**): If a sequence X exceeds 20 context tokens, truncate it from the left to keep only the most recent 20 words. This prevents long run-on sentences from bloating batch memory.
- Print one worked example displaying the raw text, cleaned tokens, and all

generated (X, y) pairs (including a truncated pair if sequence length exceeds 20).

Task 1.3: Dynamic Padding and PyTorch DataLoaders [3 Marks]

- Implement a custom **collate_fn** that handles a batch of raw, variable-length (X, y) pairs:
  - Sort the batch of pairs by sequence length **len(X)** in descending order (required for **pack_padded_sequence**).
  - Create a **lengths** tensor containing the true sequence lengths.
  - Convert each X sequence to a LongTensor, and pad using **torch.nn.utils.rnn.pad_sequence(..., batch_first=True, padding_value=0)** to output a padded tensor X with dimensions [Batch Size by Sequence Length].
  - Stack target labels into tensor Y.
  - Return the 3-tuple: **(X_padded, Y, lengths)**.
- Split your dataset into 80% training data and 20% testing data using a fixed random seed (**random_state=42**).
- Instantiate PyTorch **DataLoader** objects for the train and test sets with **batch_size=128** and your custom **collate_fn**.
- Print the shape of X, Y, and **lengths** for one single batch.

Section 2: Model Architectures, Embedding Experiments, and Training [30 Marks]

Task 2.1: Baseline Model — Vanilla RNN with Frozen GloVe [5 Marks]

- Implement **BaselineRNN** taking **vocab_size**, **embed_dim=50**, and **hidden_dim=64**.
- Initialize an **nn.Embedding(vocab_size, 50)** table with 50-dimensional GloVe vectors (**glove-wiki-gigaword-50** via Gensim or manual loading). Freeze this layer (**requires_grad=False**).
- Pack the padded sequences using **pack_padded_sequence(..., batch_first=True, enforce_sorted=True)**.
- Pass packed tensors into **nn.RNN(input_size=50, hidden_size=64, batch_first=True)**.
- Extract the final hidden state **h_n[-1]** to get the last valid timestep for each sequence.
- Apply **nn.Dropout(0.2)** followed by a Linear Layer of 64 hidden_dim to yield output logits.
- Print the model architecture and separate counts for trainable vs. non-trainable parameters.

Task 2.2: Generic Training & Evaluation Pipeline [5 Marks]

- Implement a generic training engine function: **train_model(model, model_name, train_loader, test_loader, epochs=10, lr=0.002)**.
- Train using **nn.CrossEntropyLoss(ignore_index=0)** and the Adam optimizer.
- Evaluate on **test_loader** under **torch.no_grad()** after each epoch.
- Compute metrics per epoch:
    - Testing Accuracy: Percentage of correctly predicted target words across all test samples.
    - Perplexity (PPL): e to the power of (Mean Test Loss)
    - Print the Loss, Accuracy, and PPL .
- Plot the graph of accuracy vs PPL

Task 2.3: Variant 1 — Vanilla RNN to LSTM, then Frozen to Fine-Tuned [4 Marks]

- Run 1a: Replace **nn.RNN** with **nn.LSTM(50, 64, batch_first=True)** keeping GloVe embeddings frozen (**requires_grad=False**). Train for 10 epochs.
- Run 1b (Fine-Tuning Isolation): Take the exact LSTM architecture from Run 1a and allow embedding weights to fine-tune (**requires_grad=True**). Train for 10 epochs.
- Print trainable vs. non-trainable parameter counts for both runs.

Task 2.4: Variant 2 — Domain Adaptation with Custom PyTorch CBOW [8 Marks]

Instead of relying on external pre-trained weights, build a Continuous Bag-of-Words (CBOW) model from scratch.

- CBOW Dataset Generation (with Edge Padding):
  - For every cleaned sentence, pad both ends with two tokens: **[pad,pad] + sentence + [pad,pad]**.
  - Slide a context window across the padded sentence. For every target word w_t, extract a 4-word context: [w_t-2, w_t-1, w_t+1, w_t+2].
  - This ensures start-of-sentence words (e.g., "Alice", "The") are not omitted from learning. Create a dedicated DataLoader for CBOW.

- CBOW Architecture: Implement **CBOWModel(vocab_size, embed_dim=50)**:
  - Pass context words through an **nn.Embedding(vocab_size, 50)** layer.
  - Averaging Step: Compute the mean across context vectors (**torch.mean(dim=1)**).
  - Pass the averaged vector through **nn.Linear(50, vocab_size)** to output logits.

- Training & Extraction: Train CBOW for 5 epochs. Extract the trained input weight matrix W1 (**cbow_model.embedding.weight.detach().clone()**).
- Integration: Instantiate a new LSTM model identical to Run 1a, inject the learned CBOW weights into its **nn.Embedding** layer, and freeze it (**requires_grad=False**).
- Train for 10 epochs and print loss, accuracy, and PPL.

Task 2.5: Variant 3 — Standard LSTM with Custom Self-Attention [8 Marks]

- Custom Self-Attention Module: Implement a custom **SelfAttention** module:
  - Unpack: Use **pad_packed_sequence(..., batch_first=True)** to extract output hidden states H with dimensions [Batch Size by Sequence Length by 64] for all timesteps.
  - Raw Scores: Compute raw attention scores S = Linear(H) mapping 64 dimensions to 1. The output S should have dimensions [Batch Size by Sequence Length by 1].
  - Dynamic Padding Masking: Retrieve the current batch's max sequence length T_batch = H.size(1). Create a boolean mask of shape [Batch Size, T_batch] indicating padding indices where the current timestep is greater than or equal to the sequence's true length. Fill raw scores at masked positions with **-1e9**.
  - Softmax Weights: Compute attention weights A = **softmax(S, dim=1)**.
  - Context Vector: Compute context vector C by multiplying each hidden state H by its attention weight A, and then summing across the sequence length. The output C should have dimensions [Batch Size by 64].
- Training: Pass C through **nn.Dropout(0.2)** and **nn.Linear(64, vocab_size)**. Train for 10 epochs using your custom CBOW embeddings.

Section 3: Inference and Performance Evaluation [10 Marks]

Task 3.1: Autoregressive Text Generation Pipeline [5 Marks]

- Implement a function **generate(model, seed_text, n_words=10, is_attention=False)** to generate sentences token-by-token.
- Token Processing: Clean and tokenize **seed_text**. Truncate context to the last 20 tokens if needed.
- Autoregressive Loop (10 steps):
  - Convert token IDs to a tensor and pass through the model.
  - Logit Masking: Explicitly force logits for (0) and (1) to **-1e9** to prevent non-lexical outputs.
  - Pick next token using **torch.argmax(logits, dim=1)**.
  - Append token ID to context and repeat.

Task 3.2: Deliverables, Visualizations & Analysis [5 Marks]

1. Text Generation: Generate 10-word completions for all four models (Baseline RNN, LSTM+GloVe, LSTM+CBOW, LSTM+Attention) using seeds:
   - "the white rabbit"
   - "alice was beginning"
   - "said the hatter"

2. Attention Visualization: For the LSTM+Attention model, intercept attention weights A on the seed phrase "the white rabbit". Plot a bar chart showing the weight assigned to each seed word when predicting the first target word.